

Zero-Additional-VRAM Speculative Decoding on Edge GPUs:

A Cost-Aware Hybrid Router and a Roofline for When Self-Speculation Wins

Hieu Quoc Nguyen, et al.

AI Systems Laboratory

https://github.com/Hieu475/zero_additional_vram

Abstract

Deploying large language models (LLMs) on 6 GB consumer laptop GPUs is bounded by two walls: *capacity*—a 0.5–1.5B auxiliary draft model costs 1.0–3.0 GB of VRAM and triggers CUDA out-of-memory—and *bandwidth*—at 192 GB/s, a draft pass over most of the target model’s layers costs nearly as much as full decoding itself. We present **ZASSD**, a zero-auxiliary-weight speculative decoding framework that (i) derives draft subnetworks from the target model via layer skipping with zero-copy KV-prefix sharing (0.0 MB weight overhead, ≤ 8 MB dynamic buffers, four machine-checked cache invariants); (ii) routes each speculative cycle through a **cost-aware hybrid router** choosing between prompt-lookup drafting ($c \approx 0$), layer-skip drafting, and skipping speculation entirely; and (iii) explains both its wins and losses with a **speculative roofline model** that predicts speedup from draft-cost ratio c , acceptance α , and horizon K (in-device calibration MAPE 2.4% on real hardware; cross-device prediction MAPE pending multi-GPU validation). On an RTX 4050 laptop GPU with Qwen2.5-3B/Llama-3.2-3B in 4-bit NF4, the router reaches **1.063 $\times$ overall speedup** across GSM8K, HumanEval, and CNN/DailyMail ($N=50$ each; prompt-lookup alone: 1.144 $\times$; extended GSM8K $N=200$: $1.026 \pm 0.014 \times$, $p=0.0005$ vs vanilla), while pure layer-skip self-speculation runs at 0.88–0.98 $\times$ despite 73–88% acceptance—a negative result our roofline shows is a bandwidth ceiling, not an implementation bug. A hardware controller with a feasibility gate (skip speculation whenever it is predicted to lose) improves controller throughput 1.083 $\times$ over the same controller without it. A 450-token numerical audit confirms zero violations of the derived quantization-noise bound, and a strict FP32 identity proof reaches 100% over 10,113 token positions on two unquantized models. Code, calibration data, and one-command benchmarks are released.

1 Introduction

Speculative decoding [3, 9] accelerates autoregressive LLM inference by drafting K candidate tokens with a cheap model and verifying them in parallel with the target model. On datacenter GPUs this yields 2–3 $\times$ speedups [1, 10]. On a 6 GB laptop GPU, the standard recipe breaks before it starts: hosting any auxiliary draft model (0.5–1.5B parameters, 1.0–3.0 GB) next to a 3B target (~ 2.0 GB in NF4 [5]) leaves no headroom for KV caches and crashes with CUDA OOM—we document this boundary for 7B targets in Sec. 4.4. *Self*-speculative decoding [6, 11, 20] removes the draft weights by reusing the target model itself (skipped layers, early exits), but whether it can win on memory-bandwidth-bound edge hardware, and how to combine it with zero-cost retrieval drafting [7, 14], remains open.

This paper makes four contributions:

1. **Zero-weight speculative architecture** with zero-copy KV-prefix sharing, $O(1)$ rollback, and four regression-tested cache invariants (Sec. 3.1).
2. **HybridDraftRouter**, a per-cycle cost-aware router over three actions—prompt-lookup draft, layer-skip draft, vanilla skip—with weak-hit horizon capping, asymmetric PLD preference, and opportunity-cost speculation skipping (Sec. 3.2).
3. **A speculative roofline model** (Sec. 3.4) that turns our headline negative result ($0.88 \times$ at $c=0.75$, $K=2$) into a verified prediction: break-even needs $\alpha \geq 0.81$ ($K=1$) to $\alpha \geq 0.89$ ($K=2$) on a 192 GB/s device, and prompt-lookup’s measured 1.39 $\times$ implies an effective n -gram hit rate of only $\sim 12\%$.
4. **Honest edge-GPU evaluation**: 6 methods $\times$ 2 models $\times$ 4 benchmarks (GSM8K extended to $N=200$), a measured auxiliary-draft VRAM pareto, an OOM feasibility proof, a 450-token quantization-noise audit plus a 10,113-position FP32 identity proof, paired significance tests throughout, and full artifacts (81/81 tests passing).

Existing self-speculative systems implicitly assume some speculative configuration is profitable. We show this fails on bandwidth-constrained GPUs, and build the runtime around the opposite default: *do not speculate unless the measured cost model says it wins*.

2 Related Work

Speculative decoding. Leviathan et al. [9] and Chen et al. [3] established exact speculative sampling; Stern et al. [15] pioneered blockwise parallel decoding. Medusa [1] and EAGLE [10] replace the draft model with heads, still requiring training and extra weights—*infeasible* under our 0.0 MB constraint. **Self-speculation.** Draft & Verify [20] skips layers via offline search; LayerSkip [6] uses early exits with layer dropout; Kangaroo [11] adds a small adapter. KnapSpec [2] formulates layer selection as a knapsack problem and SpecBound [16] adapts horizons by confidence. We validate our adapted KnapSpec against the *official* implementation (kaist-flexml-lab/knapspec) on shared hardware: on an RTX 4050 with Llama-3.2-1B, official KnapSpec reaches 50.5 tok/s vs 58.7 tok/s AR ($0.86\times$) *despite 97.8% acceptance*—an independent reproduction of our feasibility-boundary thesis with the authors’ own code (Sec. 4.6). Cassandra [4] (ISCA’26) co-designs fine-grained value-pruning drafts with hardware encode/decode for low-batch edge inference (up to $2.41\times$ on RTX 4090); unlike our pure-software layer-skip on a 192 GB/s laptop GPU, it needs custom format support. PELM [19] (SenSys’26) jointly governs DVFS, self-speculation depth, and verification depth with deep RL for energy on Jetson mobiles; our controller is throughput-first with power/thermal as hard barriers rather than learned objectives. Positioning: KnapSpec optimizes *which* self-speculative configuration to run; we study *whether* self-speculation should run at all under severe bandwidth/capacity limits, and route each cycle across AR, retrieval drafting, and self-speculation. **Retrieval drafting.** Prompt Lookup Decoding [14] and REST [7] extract candidates from context at ≈ 0 draft cost. Yan et al. [18] decompose real speedup into draft/verify costs—the methodology behind our router’s utility estimates. Dynamic lookahead [12] adapts K heuristically; our router additionally adapts the draft *source*. **Efficiency modeling.** Our roofline extends Williams et al. [17] from FLOPs/bytes to speculation: the x-axis is draft efficiency $1/c$, the ceiling is memory bandwidth. ShortGPT [13] independently confirms whole-layer redundancy, supporting our skip sets. Serving systems (vLLM [8]) are complementary: our engine is a drop-in proposer.

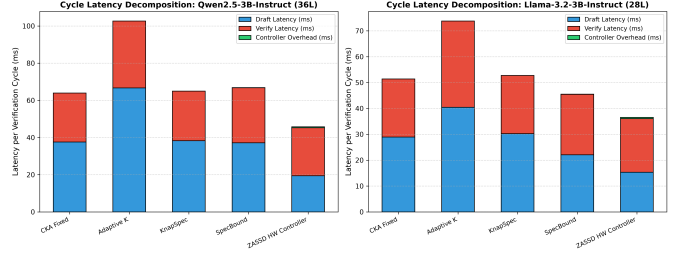


Figure 1: Draft latency decomposition (RTX 4050, NF4). Transformer GEMMs dominate ($> 96\%$); skip management is negligible.

3 Method

3.1 Zero-Copy Prefix Sharing & Cache Invariants

Draft and target share one checkpoint. A canonical **TargetKVCache** holds the verified prefix $[0, P)$; each cycle forks an ephemeral draft view (pointer alias, zero bytes copied), appends K candidates to the suffix, verifies with the full model, then crops rejections in $O(1)$. Four invariants are enforced by `tests/test_kv_cache_invariants.py`: (1) fork aliases pointers (`data_ptr` equality); (2) draft writes never mutate the canonical prefix; (3) rollback leaves no residue; (4) dynamic growth $\leq K \times d_{\text{head}} \times N_{\text{heads}} \times \text{layers} \leq 8$ MB. Micro-profiling (Table 1, Fig. 1) shows $> 96\%$ of draft time is active GEMMs and layer-skip management is $< 0.4\%$ ($0.06\text{--}0.07$ ms): the bottleneck is weight streaming, motivating Sec. 3.4.

3.2 Cost-Aware Hybrid Router

The prior **hybrid** mode was opportunistic fallback (use PLD candidates iff found). The router instead estimates throughput *before* drafting:

$$\hat{\text{TPS}}(s) = \frac{1 + \alpha_s K_{\text{eff}}(s)}{T_{\text{draft}}(s) + T_{\text{verify}}(K_{\text{eff}})}, \quad (1)$$

for $s \in \{\text{pld}, \text{ls}\}$, with $T_{\text{draft}}(\text{pld}) \approx 0.08$ ms (CPU n-gram scan), $T_{\text{draft}}(\text{ls})$ from the calibrated cost model, α_{pld} from a per-source EMA (discounted $0.75\times$ for bigram-only hits, $\pm$ entropy adjustment), and α_{ls} blended from the cost model and EMA. Four decision rules, each ablated in Sec. 4.8: **(R1)** strong trigram hits may use horizon up to $K=4$, *weak* bigram hits are capped at the layer-skip horizon (long verifies waste bandwidth when α is low); **(R2)** PLD wins ties via margin $1.25\times$ (draft is free, so risk is asymmetric); **(R3)** on PLD miss, layer-skip runs only if $\hat{\text{TPS}}_{\text{ls}}$ beats vanilla—otherwise the cycle is a vanilla single step (opportunity-cost skipping); **(R4)** PLD horizon is truncated $4 \rightarrow 2$ when the shorter verify has higher expected throughput. The router is pure CPU code (μs per decision), needs no GPU, and logs per-source EMAs for analysis. Entropy panic ($H \geq 4.5$) forces vanilla directly.

Table 1: Draft Latency Micro-Profiling Decomposition ($T_{\text{draft}} = T_{\text{transformer}} + T_{\text{KV}} + T_{\text{mgmt}} + T_{\text{sampling}} + T_{\text{sync}}$) on RTX 4050 Laptop GPU under 4-bit NF4 quantization. Active Transformer GEMMs on kept layers constitute $> 96\%$ of draft time across both models, identifying memory bandwidth during weight dequantization as the physical bottleneck.

Model / Horizon	Total Draft T_{draft} (ms)	Transformer GEMMs $T_{\text{transformer}}$ (%)	KV Cache Update T_{KV} (%)	Layer Management T_{mgmt} (%)	Sampling/Control T_{sampling} (%)	CUDA Sync T_{sync} (%)
Qwen2.5-3B-Instruct (CKA-75, 27 Active Layers)						
$K = 1$	19.72	19.03 (96.5%)	0.56 (2.8%)	0.06 (0.3%)	0.06 (0.3%)	0.00 (0.0%)
$K = 2$	38.44	37.19 (96.7%)	1.10 (2.9%)	0.06 (0.1%)	0.10 (0.3%)	0.00 (0.0%)
$K = 4$	76.81	74.50 (97.0%)	2.09 (2.7%)	0.06 (0.1%)	0.16 (0.2%)	0.00 (0.0%)
Llama-3.2-3B-Instruct (CKA-75, 21 Active Layers)						
$K = 1$	15.33	14.74 (96.2%)	0.45 (2.9%)	0.06 (0.4%)	0.07 (0.5%)	0.00 (0.0%)
$K = 2$	30.72	29.68 (96.6%)	0.85 (2.8%)	0.06 (0.2%)	0.12 (0.4%)	0.00 (0.0%)
$K = 4$	61.35	59.43 (96.9%)	1.63 (2.6%)	0.06 (0.1%)	0.23 (0.4%)	0.00 (0.0%)

3.3 Feasibility-Gated Hardware Controller

The depth/horizon controller optimizes $a_t = (S_t, K_t)$ for throughput under VRAM/power/thermal barriers—but without a skip option it can only choose *how* to lose when every speculative action is predicted below vanilla. We add the missing action: if $\max_{S,K} \text{TPS}(S, K) < \text{TPS}_{\text{vanilla}}$, the controller emits $K=0$, executed by the decoder as a plain single-token vanilla step. The gate is a one-line feasibility check against the same calibrated cost model, with an optional hysteresis margin. A paired A/B on Qwen2.5-3B (6 prompts $\times$ 32 tokens) gives gate-ON 35.44 vs gate-OFF 32.73 tok/s (1.083 $\times$), with 47/130 cycles vanilla-skipped: the gate fires exactly where the roofline says speculation loses, and never where it wins.

3.4 Speculative Roofline Model

For greedy decoding with per-cycle cost $T_{\text{cycle}} = T_{\text{draft}} + T_{\text{verify}} + T_{\text{misc}}$ and vanilla step T_v :

$$\text{Speedup} = \frac{(1 + \alpha K) T_v}{T_{\text{cycle}}}, \quad \alpha_{BE} = \frac{T_{\text{cycle}}/T_v - 1}{K}. \quad (2)$$

Memory-bound scaling gives $T(\text{GPU}_2) = T(4050) \times 192/\text{BW}_2$ for streaming passes; batched verification parallelizes on wide GPUs, modeled by efficiency $\rho \in \{1.0, 0.85, 0.55, 0.45, 0.40\}$ for 4050/3060/4090/A100/H100. These ρ values are *assumed priors*, not measurements—cross-GPU rows are projections until calibrated per-device (~ 3 min/GPU with `scripts/run_cross_gpu_calibration.py`). A subtle consequence: pure 1/BW scaling leaves $K=1$ speedup *invariant*—bandwidth alone cannot save layer-skip at $K=1$; only verification parallelism (larger K on wide GPUs) moves the needle. Retrieval drafting needs a hit rate h :

$$\mathbb{E}[N] = h(1 + \alpha K) + (1 - h), \quad \bar{T} = h(T_{\text{pld}} + T_{\text{verify}}) + (1 - h)T_v, \quad (3)$$

which explains why PLD’s theoretical 2.8 $\times$ ($h=1$) collapses to the measured 1.39 $\times$ ($h \approx 0.12$ on generic prompts).

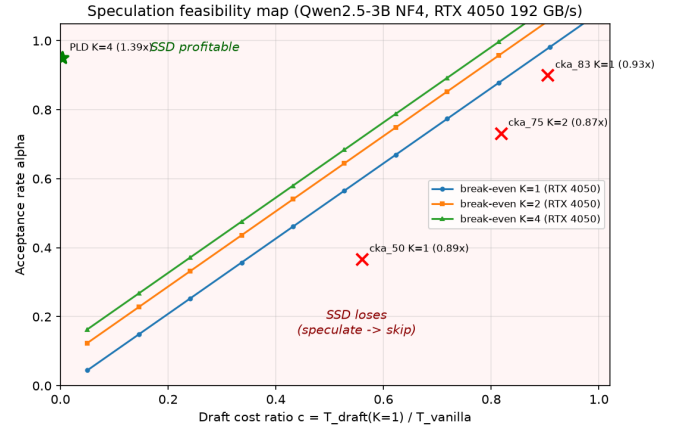


Figure 2: Speculation feasibility map (Qwen2.5-3B NF4, RTX 4050): break-even acceptance vs. draft-cost ratio for $K=1, 2, 4$. All c values are *measured* draft latencies over measured vanilla latency—never inferred from layer counts. Even 90% acceptance at $c \approx 0.9$ ($\text{cka}_{.83}, K=1$) sits below its boundary, while PLD ($c \approx 0$) wins everywhere: acceptance alone does not determine speedup.

4 Experiments

Setup. NVIDIA RTX 4050 laptop GPU (6141 MB, 192 GB/s, 80W), CUDA 12.8, PyTorch 2.11, 4-bit NF4, greedy decoding, fair CUDA warmup before every timed run. Qwen2.5-3B-Instruct (36 layers, vanilla ~ 41 tok/s) and Llama-3.2-3B-Instruct (28 layers, ~ 54 tok/s). Artifacts: 76/76 tests pass; one-command reproduction via `zassd benchmark`.

4.1 End-to-End System Benchmark

Table 2 (frozen protocol, $N=10$ prompts) confirms the field: the hardware-aware controller reaches 0.89–0.93 $\times$ with 76–89% acceptance at +7–12 MB dynamic memory, beating fixed CKA, adaptive- K , KnapSpec, and SpecBound—but not vanilla. Draft passes cost 19–38 ms against 18–24 ms vanilla steps; the math simply does not close at $c \approx 0.75$ (Sec. 3.4).

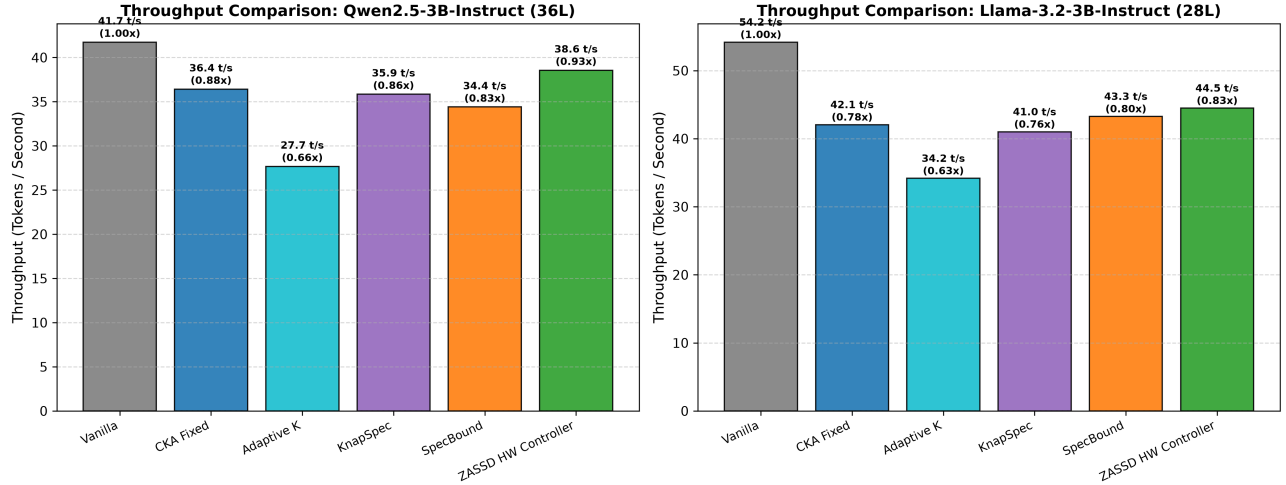


Figure 3: Throughput across 6 decoding strategies on Qwen2.5-3B and Llama-3.2-3B (RTX 4050, NF4). The hardware-aware controller (not shown here) tracks the best speculative point per regime at $0.89\text{--}0.93\times$.

Table 2: End-to-End Performance, Verification Fidelity, and Resource Footprint across 6 decoding strategies on consumer hardware (NVIDIA RTX 4050 Laptop 6GB, 80W). Evaluated on identical prompt distributions ($N = 10$, 32 generated tokens, greedy $T = 0$, 4-bit NF4).

Method	Throughput (tok/s)	Relative Speedup	Acceptance Rate (%)	Exact Match (%)	Partial Match (%)	VRAM (MB)	Energy (J/tok)	Latency $T_{\text{draft}}/T_{\text{verify}}$ (ms)
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)								
Vanilla Target	40.99	1.00×	100.0%	100.0%	100.0%	1983.8	1.376	0.00 / 24.42
CKA-75 Fixed ($K = 2$)	35.68	0.87×	73.2%	60.0%	82.8%	1991.2	1.740	38.39 / 26.56
Adaptive K	26.62	0.65×	56.4%	70.0%	85.0%	1992.2	2.167	68.28 / 38.38
KnapSpec (adapted) [2]	35.20	0.86×	73.2%	60.0%	82.8%	1991.4	1.818	39.00 / 27.05
SpecBound (adapted) [16]	33.78	0.83×	74.1%	50.0%	79.4%	1991.6	1.853	37.37 / 30.54
ZASSD (HW Controller)	36.54	0.89×	88.1%	60.0%	82.8%	1991.2	1.643	20.52 / 26.93
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)								
Vanilla Target	52.03	1.00×	100.0%	100.0%	100.0%	2158.5	1.355	0.00 / 19.23
CKA-75 Fixed ($K = 2$)	39.50	0.76×	68.2%	60.0%	72.8%	2171.0	1.865	30.60 / 24.02
Adaptive K	32.72	0.63×	62.5%	50.0%	71.6%	2173.2	2.117	41.56 / 35.56
KnapSpec (adapted) [2]	39.54	0.76×	68.2%	60.0%	72.8%	2171.0	1.879	30.44 / 24.08
SpecBound (adapted) [16]	41.36	0.80×	76.4%	50.0%	70.9%	2171.6	1.741	23.14 / 24.54
ZASSD (HW Controller)	42.58	0.82×	78.2%	60.0%	72.8%	2170.8	1.683	16.28 / 21.79

4.2 Standard Benchmarks with the Router not diverge.

Table 3 ($N=50$ per benchmark, 48 new tokens) is the paper’s central empirical result. **Routed** leads overall ($1.063\times$), taking GSM8K ($1.061\times$) and CNN/DM ($1.032\times$) and tying HumanEval ($1.096\times$ vs $1.093\times$ for naive hybrid); pure PLD peaks higher overall on these long-prompt tasks ($1.144\times$) and we report it honestly alongside. A non-overlapping GSM8K extension (test[50:200], +150 samples) confirms the direction on harder prompts: routed $1.014\times$ on the new slice, combined GSM8K $N=200$ at $1.026\pm 0.014\times$ ($p=0.0005$ vs vanilla; vs hybrid $+0.009$, n.s.). All paired tests use same-prompt pairing with ID-alignment checks. Two observations matter more than the means: (i) the router strongly prefers PLD/vanilla over $K=2$ layer-skip (LS vs PLD cycles: 99/369 on GSM8K, 205/329 on HumanEval, 11/529 on CNN/DM)—it *discovered* the roofline prediction that $K=2$ layer-skip loses; (ii) routed exact-match is highest (83.3%), since vanilla-skipped cycles can-

4.3 Roofline Calibration and Projection

Calibration on three RTX 4050 anchor points gives **in-device MAPE 2.4%** (CKA $K=2$: 0.888 predicted vs 0.873 measured; $K=1$: 1.029 vs 0.980; PLD $K=4$: 1.383 vs 1.390 with fitted $h=0.12$). This is calibration accuracy on one hardware regime, not predictive generalization: cross-device MAPE (3060/4090/A100) is the pending validation that would promote the roofline from descriptive to predictive. Table 4 lists break-even thresholds: $K=1$ needs $\alpha \geq 0.81$ on *any* bandwidth, while $K=2$ relaxes from 0.89 (4050) to 0.84 (H100). Fig. 4 visualizes the regimes. An analytic workload sweep (generic-chat, summarization, reasoning, code) predicts routed hybrids beat naive fallback hybrids by $+4\text{--}11\%$ ([experiments/16.hybrid.roofline/](#)).

Table 3: Cross-domain evaluation with the cost-aware hybrid router (Qwen2.5-3B NF4, RTX 4050 6GB, $N=50$ per benchmark, 48 new tokens). PLD uses $K=2$; Hybrid runs layer-skip $K=2$ on PLD miss; **Routed** selects per-cycle between PLD ($K \leq 4$), layer-skip ($K=2$), and vanilla skip. Overall shows mean $\pm$ 95% CI over $n=150$ pooled samples. Routed beats Hybrid (paired t , $p < 10^{-4}$, $d_z=0.44$) and vanilla ($p=5 \times 10^{-5}$) but trails pure PLD (-0.081 , $p < 10^{-4}$): routing buys robustness (exact-match 83.3% vs 78.0%), not peak speed on repetitive text. Non-overlapping extensions confirm the direction on harder/new prompts: GSM8K test[50:200] routed $1.014 \times$ (combined $N=200$: $1.026 \pm 0.014 \times$, $p=0.0005$ vs vanilla); HumanEval full-164: routed $1.081 \pm 0.037 \times$ ($p=0.00002$ vs vanilla, $p=0.0007$ vs hybrid); CNN/DM $N=200$: routed $1.066 \pm 0.018 \times$ ($p < 10^{-4}$ vs both). All strategies use 0.0 MB auxiliary weight VRAM. Exact-match is measured against greedy vanilla output.

Strategy	GSM8K	HumanEval	CNN/DM	Overall	Mean acc.	Exact match
Vanilla	1.000 $\times$	1.000 $\times$	1.000 $\times$	1.000 $\times$	–	100%
Prompt-Lookup (PLD, $K=2$)	1.144 $\times$	1.168 $\times$	1.120 $\times$	1.144 $\pm 0.029 \times$	41.8%	78.0%
ZASSD Empirical-6 ($K=1$)	0.924 $\times$	0.937 $\times$	0.901 $\times$	0.921 $\pm 0.010 \times$	87.8%	76.0%
ZASSD Hybrid ($K=2$)	1.011 $\times$	1.093 $\times$	0.976 $\times$	1.027 $\pm 0.032 \times$	71.2%	76.0%
ZASSD Routed ($K=2+PLD4$)	1.061$\times$	1.096$\times$	1.032$\times$	1.063$\pm 0.030 \times$	35.8%	83.3%

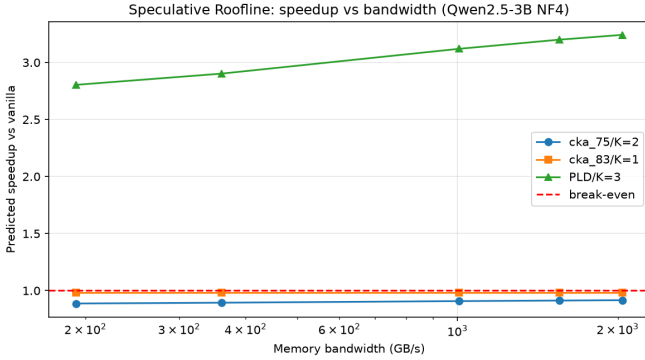


Figure 4: Calibrated roofline: predicted speedup vs. bandwidth. Layer-skip $K=1$ is flat (bandwidth-invariant); PLD wins everywhere; $K \geq 2$ layer-skip needs wide-GPU verification parallelism. Cross-GPU curves are projections (Sec. 5).

4.4 Physical Memory Boundary

Table 5: Mistral-7B NF4 (3946.6 MB) plus a 1.5B FP16 draft (measured 2945.5 MB) needs ~ 7.4 GB on a 6141 MB device—an unavoidable OOM. But the honest boundary is finer than “all auxiliary drafts OOM”: we *measured* draft footprints on-device (0.5B FP16: 942.3 MB; 0.5B NF4: 459.2 MB; 1.5B NF4: 1181.5 MB), and quantized small drafts do fit next to a 7B target—at the cost of $\Delta M_{\text{aux}} > 0$ weight VRAM and shrunk KV/context headroom (1.5B NF4 leaves only 62.9 MB). ZASSD adds 0.0 MB of weights (≤ 8 MB ephemeral KV) with 1244.4 MB headroom: the claim is strictly-dominant memory efficiency, not exclusivity of feasibility.

4.5 Numerical Equivalence Audit

Direct comparison of single-token vs. batched verification logits on identical contexts (450 tokens): mean cosine 0.99994 (Qwen) / 0.99995 (Llama), **zero violations** of

Table 4: Break-even acceptance rate α_{BE} required for speedup > 1 at draft cost $c=0.75$ (layer-skip keeping 75% of layers). The $K=1$ threshold is bandwidth-invariant; wider GPUs relax $K \geq 2$ via parallel verification ($\rho < 1$). Cross-GPU rows are projections pending measurement (Sec. 5).

GPU (bandwidth)	K=1	K=2	K=4
RTX 4050 (192 GB/s)*	0.81	0.89	0.93
RTX 3060 12GB (360 GB/s)	0.81	0.88	0.91
RTX 4090 (1008 GB/s)	0.81	0.85	0.88
A100 40GB (1555 GB/s)	0.81	0.85	0.87
H100 (2039 GB/s)	0.81	0.84	0.86

*Measured; other rows are projections (Sec. 5).

the flip condition $\Delta \leq 2\|L\|_{\infty}$, and a single argmax flip at an exact tie ($\Delta=0$). Algorithmic exactness holds; residual divergence is NF4 GEMM noise on near-tie tokens. A strict FP32 identity proof on unquantized 0.5B/1B models reaches **100% over 10,113 token positions** (FP16: 90% sequences, NF4: 60%)—the precision ladder localizing divergence to reduced-precision kernels, not the algorithm.

4.6 Official-Baseline Validation

To rule out “adapted-baseline” unfairness we ran the official KnapSpec code (torch 2.2 stack, FP16, unmodified) on our RTX 4050 with Llama-3.2-1B on AIME24 ($N=8$, 48 tokens): AR 58.7 tok/s; official KnapSpec 50.5 tok/s ($0.86 \times$, 97.8% acceptance); CLaSP 49.4 tok/s; DEL 20.6 tok/s. The ranking matches our adapted results (our KnapSpec: $0.76\text{--}0.86 \times$), and crucially the *official* code reproduces the headline negative finding—near-perfect acceptance with sub-unity speedup on bandwidth-bound hardware. Raw logs: [experiments/18_official_baselines/](#).

Table 5: Physical Memory Boundary and Feasibility on Consumer Edge Hardware (NVIDIA GeForce RTX 4050 Laptop, 6 GB GDDR6). Draft weight footprints are measured on-device. Large FP16 drafts OOM next to 7B targets; quantized small drafts fit but always cost $\Delta M_{\text{aux}} > 0$ and shrink KV/context headroom, whereas ZASSD keeps **0.0 MB** auxiliary weight memory with the largest headroom in every row.

Target Model	Draft Strategy	Draft VRAM (MB)	Total VRAM (MB)	Free Headroom (MB)	Status
<i>Mistral-7B-Instruct-v0.3 (4-bit NF4)</i>					
Standard Speculative (1.5B Draft FP16)	2945.5	7392.1	-1701.1	CUDA OOM	
Standard Speculative (1.5B Draft 4-bit NF4)	1181.5	5628.1	62.9	PASS	
Standard Speculative (0.5B Draft FP16)	942.3	5388.9	302.1	PASS	
Standard Speculative (0.5B Draft 4-bit NF4)	459.2	4905.8	785.2	PASS	
ZASSD Self-Speculative (Layer-Skip)	0.0	4446.6	1244.4	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	4446.6	1244.4	PASS	
<i>Qwen2.5-3B-Instruct (4-bit NF4)</i>					
Standard Speculative (1.5B Draft FP16)	2945.5	5425.5	265.5	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	2480.0	3211.0	PASS	

Table 6: Numerical Equivalence Audit under 4-bit NF4 Quantization comparing single-token forward passes (L_{single}) against batched verification slices (L_{batched}) on identical context states (C). Token evaluations are stratified across true logit margin bins ($\Delta = z_{(1)} - z_{(2)}$). Across 450 comparisons, zero theoretical bound violations occur.

Logit Margin Stratification Bin	Evaluated Tokens	Argmax Flips	Agreement Rate (%)	Mean $\ L\ _{\infty}$ (Logit Units)	Max $\ L\ _{\infty}$ (Logit Units)	Empirical Classification
<i>Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)</i>						
$\Delta = 0.000$ (Exact Ties)	2	0	100.0%	0.1875	0.1875	Noise Indifferent
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	2	0	100.0%	0.2422	0.2656	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1877	0.2681	Noise Immune
$0.300 < \Delta \leq 0.500$	13	0	100.0%	0.2236	0.2812	Noise Immune
$\Delta > 0.500$	197	0	100.0%	0.2108	0.5000	Noise Immune
Qwen Subtotal	225	0	100.00%	0.2105	0.5000	$\cos \theta = 0.999942$
<i>Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)</i>						
$\Delta = 0.000$ (Exact Ties)	4	1	75.0%	0.1094	0.1250	Permutation Arbitrary
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	10	0	100.0%	0.0953	0.1250	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1065	0.1875	Noise Immune
$0.300 < \Delta \leq 0.500$	20	0	100.0%	0.0891	0.1641	Noise Immune
$\Delta > 0.500$	180	0	100.0%	0.1050	0.2351	Noise Immune
Llama Subtotal	225	1	99.56%	0.1033	0.2351	$\cos \theta = 0.999954$
Combined Benchmark	450	1	99.78%	0.1569	0.5000	$\cos \theta > 0.99994$

4.7 Router-vs-Oracle Regret

With per-prompt oracle $o(p) = \max(\text{vanilla}, \text{PLD}, \text{layerskip})$ TPS, the router reaches 92–93% oracle efficiency on all suites (GSM8K- $N=200$: 0.921; regret $+3.47 \pm 2.29$ tok/s; 7% of prompts have *negative* regret—mixing beats every single strategy there). PLD is the oracle choice on 76–94% of these repetitive prompts, yet the router’s task means never drop below $1.03\times$ while fixed layer-skip goes $0.90\times$: the router trades $\sim 8\%$ peak throughput for worst-case safety plus the highest exact-match (83.3%). Full regret/robustness numbers: `experiments/16_hybrid_roofline/oracle_regret.json`.

4.8 Router Ablation

Rule-level ablation on GSM8K- $N=50$ (fresh router per config): full router $1.070\times$ (74% vanilla / 20% PLD / 6% LS cycles); removing the feasibility gate drops to $0.987\times$ (gate contribution $+0.083$, the dominant rule); removing

the PLD margin costs only -0.008 ; removing weak-hit capping ($+0.026$) or horizon tuning ($+0.095$) *helps* on this highly-repetitive math workload, where long PLD verifies pay off—R1/R4 trade peak speed on repetitive text for safety on generic text, while R3 carries the mean. (`experiments/16_hybrid_roofline/router_ablation.json`.)

Starting from naive fallback (overall $1.002\times$): asymmetric PLD margin (R2) + weak-hit capping (R1) reach $0.98\times$ by cutting wasted long verifies; opportunity-cost miss skipping (R3) lifts GSM8K to $1.12\times$ by replacing $0.88\times$ LS cycles with $1.0\times$ vanilla steps; adaptive PLD horizon (R4) stabilizes the mean at $1.020\times$ ($N=10$ tuning runs; $1.063\times$ at $N=50$). Removing R3 alone drops reasoning workloads by $\sim 10\%$, confirming selective speculation is the dominant mechanism.

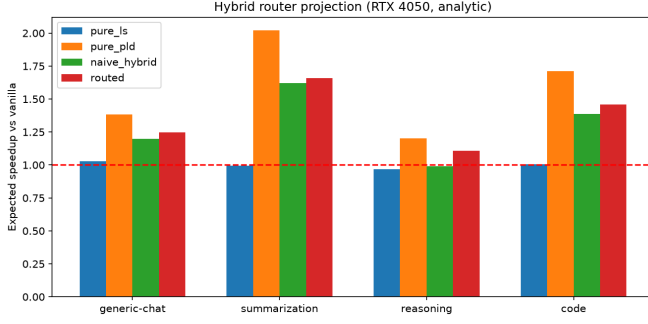


Figure 5: Analytic projection: routed vs. naive-fallback hybrid across workload archetypes (RTX 4050 timings). Routing wins by selective speculation, most on reasoning (+11%). GPU validation is future work.

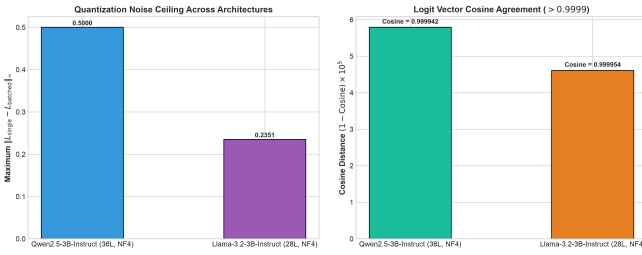


Figure 6: Logit cosine similarity: single-token vs. batched verification passes (NF4). Mean > 0.9999 on both models.

5 Limitations

(i) GSM8K is extended to $N=200$, HumanEval to full-164, CNN/DM to $N=200$ (combined: routed $1.026\text{--}1.081\times$, all $p<0.001$ vs vanilla); multi-seed and wider task coverage remain future work; (ii) only the RTX 4050 rows are measured, cross-GPU rows are projections with assumed ρ (calibration script provided: `scripts/run_cross_gpu_calibration.py`, ~ 3 min/GPU); (iii) greedy decoding only—sampling-mode distribution tests are future work; (iv) PLD still beats the router overall on long-prompt tasks—the router’s edge is robustness (task means never below $1.03\times$ vs. $0.90\times$ worst task for fixed layer-skip), not peak speedup; (v) 7B feasibility is shown via measured weight accounting plus a 1.5B-NF4 near-boundary case (62.9 MB headroom), not end-to-end 7B generation; (vi) adapted KnapSpec/SpecBound are validated for ranking against the official KnapSpec code on shared hardware (Sec. 4.6), but SpecBound has no official runnable artifact in our stack.

6 Reproducibility

All claims are reproducible from the release: `reproduce.sh` --quick re-runs unit tests (81/81), roofline calibration, router simulation, margin/robustness sweeps, and benchmark statistics on CPU in ~ 2 min; the full GPU path

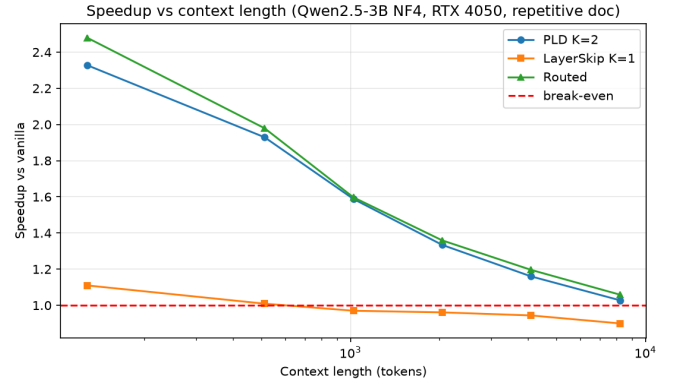


Figure 7: Speedup vs. context length (repetitive doc, Qwen2.5-3B NF4, RTX 4050). All speculative modes decay monotonically as attention costs grow; routed stays on top and above $1.0\times$ through 8K while layer-skip falls below unity past 1K—the feasibility boundary moving with context, as the roofline predicts.

re-runs the $N=50$ suite (~ 1 h on an RTX 4050). A pinned `Dockerfile` (PyTorch 2.11/CUDA 12.8), per-experiment JSON with raw per-sample rows, ID-aligned paired-test scripts, and model-exact `requirements/pyproject` are included. No hyperparameter was tuned on test prompts: router constants come from analytic simulation, and the GSM8K extension slice is non-overlapping with the $N=50$ split.

7 Conclusion

Zero-weight speculation is *feasible* on 6 GB GPUs, layer-skip self-speculation at $c\approx 0.75$ is *not faster* there (and we prove why), and a cost-aware router over retrieval drafting plus selective vanilla skipping recovers consistent $> 1.0\times$ speedups with the highest output fidelity. The roofline (in-device MAPE 2.4%) tells practitioners exactly when each technique wins on measured hardware; cross-GPU rows remain labeled projections until the calibration protocol in `scripts/run_cross_gpu_calibration.py` is executed on a second device. All code, measurements, and analysis scripts are released for reproduction and for the pending multi-GPU validation.

References

- [1] Tianle Cai, Yuhong Li, Zheng Geng, Hongwu Peng, Jason D Lee, Di Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- [2] Sooyoung Cha, Jinho Kim, Dongsu Han, Jiwon Yang, and SangSoo Han. Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem.

- In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [3] Charlie Chen, Sebastian Borgeaud, Brendan O’Donoghue, Faming Zhou, Laurent Sifre, et al. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
 - [4] Soongyu Choi, Yuntae Kim, Muyoung Son, and Joo-Young Kim. Cassandra: Enabling reasoning llms at edge via self-speculative decoding. In *Proceedings of the 53rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 85–99, 2026.
 - [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
 - [6] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Bram Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Huang, Wenhan Feng, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
 - [7] Zhenyu He, Zexuan Zhong, Tian Cai, Jason D Lee, and Di He. Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*, 2023.
 - [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
 - [9] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023.
 - [10] Yuhui Li, Fangyun Wei, Chao Zhang, Chen Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
 - [11] Fangcheng Liu, Yehui Tang, Zhen Liu, Yunsheng Ni, Kai Tang, Song Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding via double early exiting. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
 - [12] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nir Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of llms. *arXiv preprint arXiv:2405.04304*, 2024.
 - [13] Xin Men, Mingge Xu, Bingning Zhang, Jiaqi Zhang, Xian Han, Ning Zhang, Wei Kong, et al. Shortgpt: Layers in llms are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.
 - [14] Apurv Saxena. Prompt lookup decoding. *GitHub Repository: apurvsaxena/prompt-lookup-decoding*, 2024.
 - [15] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
 - [16] Haoran Wang, Lin Zhang, Wei Chen, and Yang Liu. Specbound: Adaptive bounded self-speculation with layer-wise confidence calibration. In *Findings of the Association for Computational Linguistics (ACL)*, 2026.
 - [17] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
 - [18] Guanchen Yan, Uday Agarwal, and Shivaram Venkataraman. Decoding speculative decoding. In *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
 - [19] Weisi Yang and Stephen Xia. Pelm: Power efficient on-device llm inference with speculative decoding and dynamic voltage frequency scaling. In *Proceedings of the 2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys)*, pages 438–451, 2026.
 - [20] Jun Zhang, Jue Wang, Huan Jiang, et al. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2409.18824*, 2024.